Self-Hosted Private Cloud Storage
Using Nextcloud, Docker, Ngrok and WebDAV

Project Documentation

Abstract

A self-hosted cloud storage system was designed and implemented on a Windows 11 laptop with a 512 GB SSD. Nextcloud Server 27 and MariaDB 10.6 were deployed using Docker Desktop and Docker Compose. The laptop's local storage was used to store cloud files. Ngrok provided remote access through an HTTPS tunnel, while Nextcloud user management was used to create separate user accounts. A PowerShell script using WebDAV was developed to process files in a designated cloud inbox, and Windows Task Scheduler was configured to run the script automatically. Mobile access and automatic uploads were also configured using the Nextcloud mobile application.

Introduction

Cloud storage allows users to store files on a server and access them from different devices. In this project, a personal laptop was configured to act as a private cloud storage server. Instead of depending on a third-party cloud storage provider, the implementation uses local hardware and open-source software. Docker provides an isolated environment for the application and database, while Nextcloud provides the interface for file storage, user management and access.

Motivation and Problem Statement

The purpose of this project was to understand cloud storage by implementing the complete setup on personal hardware. The main objective was to provide centralized file storage, access from multiple devices, separate user accounts and automated file handling using locally managed infrastructure.

Objectives

- Deploy Nextcloud and MariaDB using Docker containers.
- Use the laptop's SSD as the storage location.
- Provide remote access to the Nextcloud service.
- Create separate user accounts and verify individual file spaces.
- Automate file processing using PowerShell and WebDAV.
- Configure scheduled execution and mobile access.

Tools and Technologies Used

Windows 11: Used as the host operating system on a laptop with a 512 GB SSD.

Docker Desktop: Used to run Nextcloud and MariaDB in separate containers.

Docker Compose: Used to define and manage the application and database services, including their environment variables, ports and volume mappings.

Nextcloud Server 27: Provides the web interface for file storage, sharing, user management and mobile access.

MariaDB 10.6: Stores the database information required by Nextcloud.

Ngrok: Provides an HTTPS tunnel to access the locally hosted Nextcloud service remotely.

WebDAV: Used by the PowerShell script to list, copy and delete files through Nextcloud's file interface.

PowerShell: Used to create the automated file-processing script.

Windows Task Scheduler: Used to execute the PowerShell script at scheduled intervals.

System Architecture

The system consists of client devices, a remote-access layer, an application layer, a database and local storage. Users access Nextcloud through a web browser or mobile application. For remote access, Ngrok forwards HTTPS traffic to the Nextcloud service running locally on port 8081. Nextcloud and MariaDB run as Docker containers on the Windows laptop. Host directories are mapped into the containers so that application and file data remain stored on the laptop's SSD.

The Docker Compose configuration maps the database directory to `./db`, the Nextcloud application directory to `./data`, and the host folder `C:/cloud_storage` to `/cloud_storage` inside the container.

Implementation Details

Step 1: Installing Docker and Preparing the Environment

Docker Desktop was downloaded and installed on the Windows 11 system. After verifying that the Docker engine was running, a folder named `C:\nextcloud_docker` was created. This folder was used to store the Docker Compose file and persistent application data.

Step 2: Configuring Docker Compose

A file named `docker-compose.yml` was created to define two services: MariaDB 10.6 and Nextcloud Server 27. The database service was configured with a dedicated database and user. The Nextcloud service was configured to connect to the database using the Compose service name `db`. Port 8081 on the Windows host was mapped to port 80 inside the Nextcloud container.

The services were started from PowerShell using `docker compose up -d`. Nextcloud was then opened at `http://localhost:8081`, and the initial administrator account and database connection details were configured through the setup page.

Step 3: Configuring External Storage

The host folder `C:/cloud_storage` was mapped into the Nextcloud container at `/cloud_storage`. External Storage Support was enabled from the Nextcloud application settings. A Local external storage entry named `cloud_inbox` was created and configured to use the mounted path. The storage entry was made available to the administrator account.

Step 4: Configuring Multi-User Access

A separate end-user account was created through the Nextcloud administration settings. Nextcloud created an individual file directory for the new account. The project documentation verified that the administrator and the additional user had separate directories under the Nextcloud data folder. This demonstrated the user-level separation of files within the instance.

Step 2.2: Multi-user access (Isolated)

To verify the multi-user capability, I created a separate account.

Steps:

1. logged in as admin

2. navigated to admin setting --->users

3. added new end-user named ;user1

Nextcloud automatically created a dedicated folder.

Each user can only access their own files - complete isolation is maintained

Path: C:\nextcloud_docker\data\data\user1\files

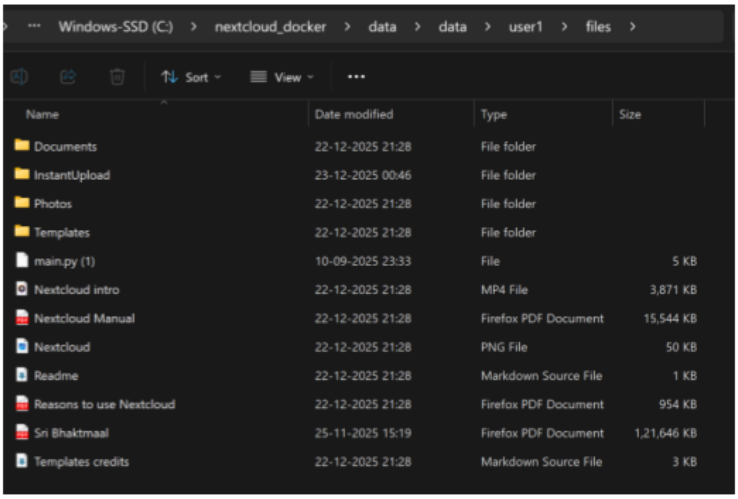


Fig: user1's dedicated space

Step3 :Getting Internet Access

My laptop is connected to wifi, so the ip address changes sometimes. Also, I wanted to access my cloud from outside my home network (WAN).

Using Ngrok for External Access

1. download ngrok from https://ngrok.com/
2. In cmd: ngrok http 8081

   Result : https://unexpectorated-letisha-geodetically.ngrok-free.dev

   Important otes:

   1. free ngrok sessions expire after 2 hours

   2. url changes every time you restart ngrok

Figure 1. Screenshot from the original documentation showing the user directory and remote-access setup.

Step 5: Configuring Remote Access Using Ngrok

The laptop's local IP address could change when connecting to Wi-Fi, and access from outside the local network was required. Ngrok was configured to forward external traffic to local port 8081. The command used during setup was `ngrok http 8081`. Ngrok generated an HTTPS endpoint for remote access.

The generated endpoint was added to the Nextcloud `trusted_domains` configuration. The `overwrite.cli.url` setting was also updated to use the HTTPS endpoint to resolve a redirect issue. The project notes indicate that the free Ngrok endpoint could expire and change after restarting the tunnel.

Step 6: PowerShell Automation Using WebDAV

A PowerShell script was created to process files placed in the Nextcloud `cloud_inbox` folder. The script connects to Nextcloud through its WebDAV endpoint and authenticates using an application password generated from the Nextcloud security settings.

The script performs the following operations:

- Sends a WebDAV PROPFIND request to list files in the cloud inbox.
- Reads the returned file paths and identifies individual files.
- Sends a WebDAV COPY request to copy each file to the administrator's main file area.
- Sends a DELETE request to remove the original inbox file after the copy request.
- Displays a completion message after processing.

This workflow automates the movement of files from the inbox to the main storage area. The project documentation records the use of a dedicated Nextcloud application password for script authentication.

Step 7: Configuring Windows Task Scheduler

Windows Task Scheduler was configured to run the PowerShell script automatically. A task was created with a program action that starts PowerShell and passes the script path as an argument. A schedule was configured so that the file-processing script could execute without requiring manual execution each time.

Setting Task scheduler:

1. Task Scheduler
2. Enter create task
3. Enter name
4. High privilege
5. Set time
6. Start a programm
7. Argument: -ExecutionPolicy Bypass -WindowStyle Hidden -File "C:\nextcloud_docker\scripts\nextcloud_ingest.ps1"
8. Done

Step5 : Mobile Access

One way:

1. LAN: The browser access using : http://10.186.74.67:8081
2. WAN: using : https://unexpectorated-letisha-geodetically.ngrok-free.dev

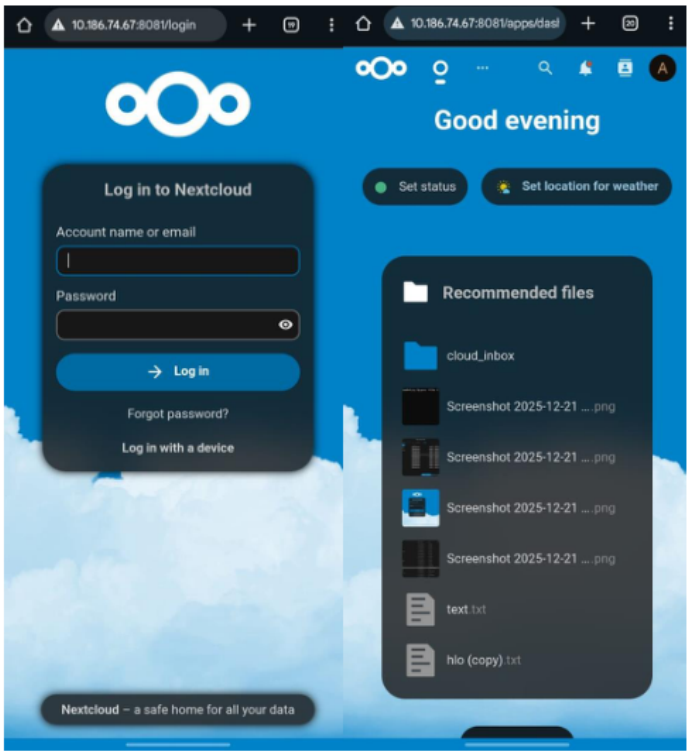


Fig:Mobile access

Another way:

1. Download nextcloud from play store
2. Choose login and enter : https://unexpectorated-letisha-geodetically.ngrok-free.dev
3. Fill credentials
4. Done
5. For-auto upload :

Figure 2. Screenshot from the original documentation showing Task Scheduler and mobile access.

Step 8: Mobile Access

The Nextcloud mobile application was configured to connect to the server using the remote HTTPS endpoint. Automatic upload was enabled for a selected folder on the mobile device. The project notes describe using the screenshots folder as the upload source. Uploaded files were visible in the corresponding user's Nextcloud storage directory.

a) Select enable auto upload
b) Select folder(I choosed screenshot)
c) Unmetered wifi

The data resides in : C:\nextcloud_docker\data\data\user1\files\InstantUpload\Screenshots

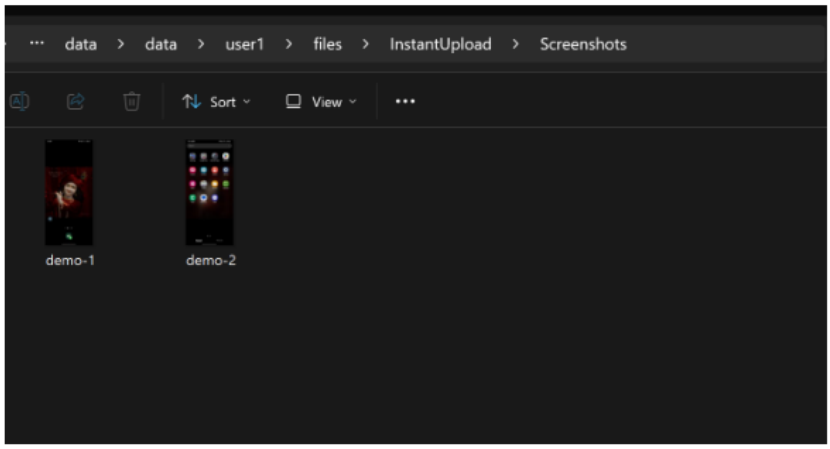


Fig: screenshot folder for user1(the another end user except admin)

Every user get isolated with their own space as proof in the screenshot, where admin and user1 have individual storage:

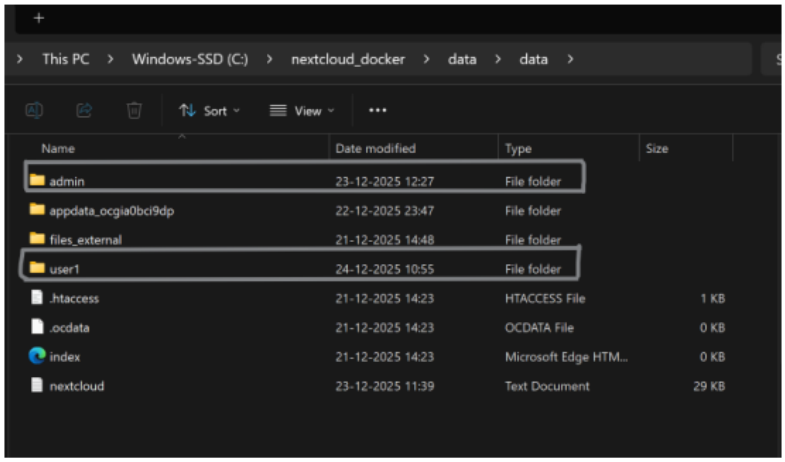


Fig: two different folders for different end users (admin and user1)

Conclusion:

I basically built my own free version of Google Drive. Now my phone photos automatically save straight to my PC's SSD. It feels good knowing that 100% of my data stays with me, giving me total privacy without depending on any big company servers.

Figure 3. Screenshot from the original documentation showing mobile-upload files and separate user directories.

Problems Encountered and Resolutions

External storage was not visible automatically: The host folder had been mounted into the container, but it was not visible in Nextcloud until External Storage Support was enabled and a Local storage entry was configured.

Remote access was required: Ngrok was used to provide an HTTPS tunnel to the local Nextcloud service.

Untrusted domain restriction: Access through the remote endpoint was initially blocked by Nextcloud's trusted-domain security check. The endpoint was added to the `trusted_domains` configuration.

Redirect issue: The `overwrite.cli.url` setting was updated to use the remote HTTPS endpoint.

Manual file processing: A PowerShell script using WebDAV was developed and configured for scheduled execution.

Results and Outcomes

- Nextcloud Server 27 and MariaDB 10.6 were successfully deployed using Docker Compose.
- Local laptop storage was connected to Nextcloud through volume mappings.
- External storage was configured and made available in the Nextcloud interface.
- A separate user account and individual file directory were created and verified.
- Remote access was configured using an Ngrok HTTPS tunnel.
- A PowerShell script was implemented to process files through WebDAV.
- Windows Task Scheduler was configured to run the automation script.
- Mobile access and automatic uploads were configured using the Nextcloud application.

Advantages

- Files are stored on locally managed hardware.
- The system provides a web interface and mobile access.
- Separate user accounts provide individual file spaces.
- File processing can be automated using WebDAV and PowerShell.
- The deployment uses open-source software and locally available resources.

Learning Outcomes

- Practical experience with Docker and Docker Compose.
- Understanding of application and database deployment.
- Experience with persistent storage and volume mapping.
- Understanding of Nextcloud administration and user management.
- Practical experience with remote access and trusted-domain configuration.
- Experience with WebDAV requests and PowerShell scripting.
- Understanding of scheduled task automation in Windows.

Conclusion

A self-hosted private cloud storage system was implemented using Nextcloud, Docker, MariaDB, Ngrok and PowerShell on a Windows 11 laptop. The project combined containerized application deployment, local storage, user management, remote access and file automation. It provided practical experience in cloud storage architecture, containerization, networking and system administration. The implementation also provides a starting point for future improvements such as persistent remote-access configuration, backup management and additional monitoring.

Note: This report is based on the supplied project documentation. The Ngrok URL and credentials are not reproduced because the endpoint is temporary and credentials should remain private. The documented automation performs a COPY request followed by DELETE; this report does not claim transactional transfer guarantees.